

1. Contexto y objetivos

El objetivo es desarrollar una aplicación web de una sola página (SPA simple, sin frameworks) que consuma una API pública, procese los datos recibidos y los muestre al usuario de forma clara, con funcionalidades de búsqueda, filtros y paginación.

Además, se introduce el uso de Git y GitHub como herramienta profesional de control de versiones, fundamental para el resto de la carrera y para el ámbito laboral.

Se debe subir el link al repositorio utilizado, deben subir el nombre de los integrantes del trabajo en un archivo de texto y subir por cada IA utilizada y cada alumno diferente un archivo (pdf, doc, texto, etc) con las preguntas hechas a la IA y lo que la misma les respondió.

2. API a utilizar: Rick and Morty API

Trabajaremos con la API pública de Rick and Morty, que provee datos sobre personajes, episodios y locaciones de la serie. Es gratuita, no requiere registro ni clave de acceso, y devuelve datos en formato JSON.

Documentación oficial: <https://rickandmortyapi.com/documentation>

Endpoints principales que vamos a usar

GET <https://rickandmortyapi.com/api/character>

Devuelve un listado paginado de personajes (20 por página).

GET <https://rickandmortyapi.com/api/character?page=2>

Navegación entre páginas usando el parámetro page.

GET <https://rickandmortyapi.com/api/character?name=rick&status=alive>

Filtros combinables: name, status (alive/dead/unknown), species, gender.

GET <https://rickandmortyapi.com/api/character/1>

Detalle de un personaje específico por ID.

Estructura de la respuesta

```
{ "info": { "count": 826, "pages": 42, "next": "...", "prev": null },
  "results": [ { "id": 1, "name": "Rick Sanchez", "status": "Alive",
                 "species": "Human", "gender": "Male", "image": "...",
                 "origin": {...}, "location": {...}, "episode": [...] } ] }
```

3. Consigna

Desarrollar una aplicación web que permita al usuario explorar los personajes de Rick and Morty consumiendo la API mencionada. La aplicación debe contar con la siguiente funcionalidad:

3.1. Listado de personajes

- Al cargar la página, mostrar los primeros 20 personajes en formato de tarjetas (cards).
- Cada tarjeta debe mostrar: imagen, nombre, estado (con indicador visual de color), especie y origen.
- El indicador de estado debe ser un círculo o badge de color: verde (Alive), rojo (Dead), gris (unknown).

3.2. Paginación

- Mostrar botones de "Anterior" y "Siguiente" para navegar entre páginas.
- Mostrar el número de página actual y el total de páginas (ej: Página 2 de 42).
- Los botones deben deshabilitarse cuando no haya página anterior o siguiente.

Requerimientos técnicos

- HTML5 semántico (uso de header, main, section, article, footer).
- CSS en archivo separado (no inline ni en <style>). Diseño responsivo (debe verse bien en celular y escritorio).
- JavaScript en archivo separado, organizado en funciones reutilizables.
- Uso de fetch() con async/await o promesas (then/catch).
- Código comentado en las partes clave.

4. Trabajo con Git y GitHub

El proyecto debe entregarse a través de un repositorio público en GitHub.

Pasos a seguir

1. Crear una cuenta en GitHub (si no la tienen ya) en <https://github.com>
2. Instalar Git en su computadora desde <https://git-scm.com/downloads>
3. Configurar Git con su nombre y email:

```
git config --global user.name "Su Nombre"
```

```
git config --global user.email "su.email@ejemplo.com"
```

4. Crear un nuevo repositorio en GitHub llamado rick-and-morty-explorer (público).
5. Clonar el repositorio en su máquina local:

```
git clone https://github.com/usuario/rick-and-morty-explorer.git
```

6. Trabajar en el proyecto haciendo commits frecuentes con mensajes descriptivos.
7. Subir los cambios a GitHub con git push.

Comandos Git esenciales

```
git status          # Ver el estado de los archivos
git add .           # Agregar todos los cambios al stage
git commit -m "mensaje" # Crear un commit con un mensaje
git push            # Subir los commits al repositorio remoto
git pull            # Traer los últimos cambios del remoto
git log             # Ver el historial de commits
```

Buenas prácticas de commits

- Hacer commits pequeños y frecuentes, no uno solo al final.
- Mensajes claros y en infinitivo: "Agregar filtro por género", no "cambios" o "asdf".
- Incluir un archivo .gitignore para excluir archivos innecesarios.

Estructura sugerida del repositorio

```
rick-and-morty-explorer/
├── index.html
├── css/
│   └── styles.css
├── js/
│   ├── api.js      (funciones que llaman a la API)
│   ├── ui.js       (funciones que manipulan el DOM)
│   └── main.js      (lógica principal y eventos)
├── img/             (imágenes propias del proyecto)
├── .gitignore
└── README.md
```

Contenido mínimo del README.md

- Título del proyecto y descripción breve.
- Nombre del o los integrantes.
- Tecnologías utilizadas.
- Capturas de pantalla (al menos 2).
- Instrucciones para ejecutar el proyecto localmente.
- Link al deploy (si se realizó).
- API utilizada y créditos.

5. Recursos de apoyo

Documentación

- MDN Web Docs (Fetch API): https://developer.mozilla.org/es/docs/Web/API/Fetch_API/Using_Fetch
- Rick and Morty API: <https://rickandmortyapi.com/documentation>
- Git Handbook (en español): <https://docs.github.com/es/get-started/using-git/about-git>
- Markdown para README: <https://www.markdownguide.org/cheat-sheet/>

Consejos finales

- Empezar por lo más simple: hacer funcionar el fetch y mostrar los datos en consola antes de pensar en el diseño.
- Probar la API directamente desde el navegador pegando la URL antes de programar nada.
- Hacer commits cada vez que se logra que algo funcione, por pequeño que sea.
- Pedir ayuda al docente cuando se traben más de 30 minutos en un mismo error.
- Usar la consola del navegador (F12) constantemente para depurar.